Evidencias Docker

Instituto Politécnico Nacional IPN

Escuela Superior de Computo ESCOM

Alumnos:

López Ramos Marcos Isaid

Unidad de aprendizaje: Internet de las cosas | Embedded systems

Grupo: 6CM3

Profesor: Aleman Arce Miguel Angel

Introducción

En esta práctica se desarrollaron contenedores Docker capaces de ejecutar programas en lenguaje C y Python, generando como salida imágenes en formato JPG y PNG respectivamente. Se documenta el proceso de instalación, configuración, ejecución y validación de los contenedores.

Instalación de Docker

Instalación de Docker con los siguientes comandos: Instalación de Docker en Ubuntu mediante la actualización del sistema, instalación de dependencias, configuración del repositorio oficial y descarga de los paquetes necesarios.

- sudo apt update

Instala dependencias:

- sudo apt install -y ca-certificates curl gnupg

Agrega la clave oficial de Docker:

- sudo install -m 0755 -d /etc/apt/keyrings
- curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo tee /etc/apt/keyrings/docker.asc > /dev/null
- sudo chmod a+r /etc/apt/keyrings/docker.asc

Agrega el repositorio:

- echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo $VERSION_CODENAME) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

Instala Docker:

- sudo apt update
- sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

```
nonbankstew@MSI:~$ sudo apt install -y ca-certificates curl gnupg
[sudo] password for nonbankstew:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20240203).
ca-certificates set to manually installed.
curl is already the newest version (8.5.0-2ubuntu10.8).
curl set to manually installed.
gnupg is already the newest version (2.4.4-2ubuntu17.4).
gnupg set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
nonbankstew@MSI:~$ sudo install -m 0755 -d /etc/apt/keyrings
nonbankstew@MSI:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo tee /etc/apt/keyrings/docker.asc > /dev/null
nonbankstew@MSI:~$ sudo chmod a+r /etc/apt/keyrings/docker.asc
nonbankstew@MSI:~$ echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo $VERSION_CODENAME) stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
nonbankstew@MSI:~$ sudo apt update
Get:1 https://download.docker.com/linux/ubuntu noble InRelease [48.5 kB]
Get:2 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [51.8 kB]
Hit:3 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:6 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Fetched 100 kB in 1s (195 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
nonbankstew@MSI:~$ sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Ejecución del Docker en Ubuntu:

- Verificación del funcionamiento de Docker mediante la ejecución del contenedor de prueba *hello-world*, confirmando la correcta comunicación entre el cliente y el daemon.

```
nonbankstew@MSI:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:f9078146db2e05e794366b1bfe584a14ea6317f44027d10ef7dad65279026885
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Para evitar usar sudo siempre:

- Configuración de permisos para permitir la ejecución de Docker sin necesidad de utilizar el comando sudo.

```
nonbankstew@MSI:~$ sudo usermod -aG docker $USER
nonbankstew@MSI:~$ |
```

Creación del Docker

Código en C:

- Código fuente en lenguaje C utilizado para generar una imagen en formato PPM mediante la manipulación de valores RGB por píxel.

```
nonbankstew@MSI: ~/tarea-c
  GNU nano 7.2                                  src/generar_imagen.c
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *archivo = fopen("output/imagen.ppm", "w");

    if (archivo == NULL) {
        printf("Error al crear la imagen.\n");
        return 1;
    }

    int ancho = 400;
    int alto = 300;

    fprintf(archivo, "P3\n");
    fprintf(archivo, "%d %d\n", ancho, alto);
    fprintf(archivo, "255\n");

    for (int y = 0; y < alto; y++) {
        for (int x = 0; x < ancho; x++) {
            int r = (x * 255) / ancho;
            int g = (y * 255) / alto;
            int b = 150;

            fprintf(archivo, "%d %d %d ", r, g, b);
        }
        fprintf(archivo, "\n");
    }

    fclose(archivo);

    printf("Imagen PPM generada correctamente.\n");
    return 0;
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo
```

Makefile:

- Archivo Makefile utilizado para automatizar la compilación, ejecución y conversión del programa en C dentro del contenedor Docker.

```
nonbankstew@MSI: ~/tarea-c
  GNU nano 7.2                                   Makefile
CC = gcc
CFLAGS = -Wall -O2
SRC = src/generar_imagen.c
BIN = generar_imagen
OUT = output/imagen.jpg

all: prepare build run convert

prepare:
        mkdir -p output

build:
        $(CC) $(CFLAGS) $(SRC) -o $(BIN)

run:
        ./$(BIN)

convert:
        convert output/imagen.ppm $(OUT)
        @echo "Imagen JPG generada en: $(OUT)"

clean:
        rm -f $(BIN)
        rm -f output/*.ppm output/*.jpg

^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute     ^C Location
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify     ^/ Go To Line
```

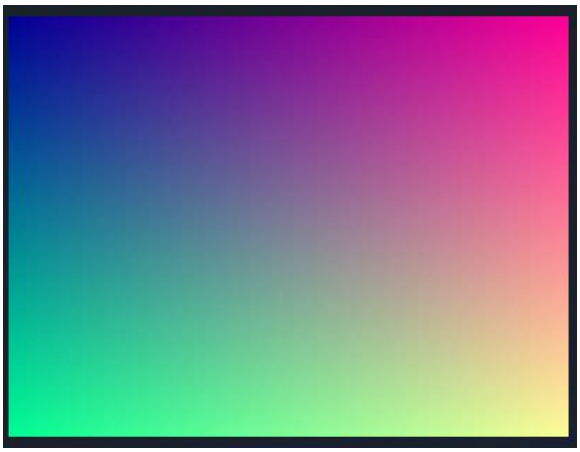
Creación:

- Proceso de construcción de la imagen Docker y ejecución del contenedor, donde se compila el programa en C y se genera la imagen final en formato JPG.

```
nonbankstew@MSI: ~/tarea-c
nonbankstew@MSI:~/tarea-docker/contenedor-c-jpg$ nano Makefile
nonbankstew@MSI:~/tarea-docker/contenedor-c-jpg$ nano src/generar_imagen.c
nonbankstew@MSI:~/tarea-docker/contenedor-c-jpg$ nano Makefile
nonbankstew@MSI:~/tarea-docker/contenedor-c-jpg$ nano Dockerfile
nonbankstew@MSI:~/tarea-docker/contenedor-c-jpg$ docker build -t contenedor-c-jpg .
[+] Building 80.4s (9/9) FINISHED                                          docker:default
 => [internal] load build definition from Dockerfile                                  0.0s
 => => transferring dockerfile: 276B                                                  0.0s
 => [internal] load metadata for docker.io/library/ubuntu:22.04                       1.4s
 => [internal] load .dockerignore                                                     0.0s
 => => transferring context: 2B                                                       0.0s
 => [1/4] FROM docker.io/library/ubuntu:22.04@sha256:962f6cadeae0ea6284001009daa4cc9a8c37e  3.3s
 => => resolve docker.io/library/ubuntu:22.04@sha256:962f6cadeae0ea6284001009daa4cc9a8c37e  0.0s
 => => sha256:f63eb04151bcac21ad049f8d781b97b219aba392c5457907f8f3e88e43 29.74MB / 29.74MB  2.6s
 => => extracting sha256:f63eb04151bcac21ad049f8d781b97b219aba392c5457907f8f3e88e43eb48ec   0.6s
 => [internal] load build context                                                     0.0s
 => => transferring context: 1.50kB                                                   0.0s
 => [2/4] RUN apt-get update && apt-get install -y     build-essential     imagemagick     60.5s
 => [3/4] WORKDIR /usr/src/app                                                        0.2s
 => [4/4] COPY . .                                                                    0.1s
 => exporting to image                                                               14.4s
 => => exporting layers                                                              11.1s
 => => exporting manifest sha256:2bd5c8f923c97c53ea4d587afaec5bbb740fe69931390d5d7c03e695d  0.1s
 => => exporting config sha256:59c1be15a79b99c25e68722091ad2a9d9f5dac789434e1006bf3521c2c1  0.1s
 => => exporting attestation manifest sha256:b518f4d33cfa0e69ae51154397a366fb0963d8f10f2c7  0.2s
 => => exporting manifest list sha256:924a492e3bb3f2ba67c486ba873d2917f1a26baea22c5b6ad666  0.1s
 => => naming to docker.io/library/contenedor-c-jpg:latest                            0.0s
 => => unpacking to docker.io/library/contenedor-c-jpg:latest                         2.7s
nonbankstew@MSI:~/tarea-docker/contenedor-c-jpg$ docker run --rm -v "$PWD:/usr/src/app" contenedor-c-jpg
mkdir -p output
gcc -Wall -O2 src/generar_imagen.c -o generar_imagen
./generar_imagen
Imagen PPM generada correctamente.
convert output/imagen.ppm output/imagen.jpg
Imagen JPG generada en: output/imagen.jpg
nonbankstew@MSI:~/tarea-docker/contenedor-c-jpg$ ls output
imagen.jpg  imagen.ppm
nonbankstew@MSI:~/tarea-docker/contenedor-c-jpg$
```

Imagen generada:

- Imagen generada por el programa en C, resultado de la conversión del formato PPM a JPG mediante ImageMagick dentro del contenedor.



Docker propio

Construcción y ejecución del contenedor adicional en Python, donde se genera una gráfica a partir de datos simulados.

```
nonbankstew@MSI: ~/tarea-c
nonbankstew@MSI:~$ cd ~/tarea-docker
nonbankstew@MSI:~/tarea-docker$ ls
contenedor-c-jpg
nonbankstew@MSI:~/tarea-docker$ mkdir -p contenedor-python-grafico/output
nonbankstew@MSI:~/tarea-docker$ cd contenedor-python-grafico
nonbankstew@MSI:~/tarea-docker/contenedor-python-grafico$ nano grafico.py
nonbankstew@MSI:~/tarea-docker/contenedor-python-grafico$ nano grafico.py
nonbankstew@MSI:~/tarea-docker/contenedor-python-grafico$ nano Dockerfile
nonbankstew@MSI:~/tarea-docker/contenedor-python-grafico$ docker build -t contenedor-python-grafico .
[+] Building 68.3s (9/9) FINISHED                                                                  docker:default
 => [internal] load build definition from Dockerfile                                                          0.0s
 => => transferring dockerfile: 156B                                                                          0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim                                           5.4s
 => [internal] load .dockerignore                                                                             0.0s
 => => transferring context: 2B                                                                               0.0s
 => [1/4] FROM docker.io/library/python:3.11-slim@sha256:6d85378d88a19cd4d76079817532d62232be95757cb45945a99fec8  16.9s
 => => resolve docker.io/library/python:3.11-slim@sha256:6d85378d88a19cd4d76079817532d62232be95757cb45945a99fec8e  0.0s
 => => sha256:7ccd73948dde4b7f1623ac6bf9b58706fb83d0a363d765d7c799d46035ceb563 249B / 249B                    0.3s
 => => sha256:f3ba2250c52436e41e9f2f3830af92de466fcacc2d848264e1f9fdc76975d803 14.37MB / 14.37MB              15.2s
 => => sha256:91ff8760033c3b9bc9c3a3bcc6ff66c113ec9e13f94fa2f12ae4b221aae996b9 1.29MB / 1.29MB                2.3s
 => => sha256:3531af2bc2a9c8883754652783cf96207d53189db279c9637b7157d034de7ecd 29.78MB / 29.78MB              15.5s
 => => extracting sha256:3531af2bc2a9c8883754652783cf96207d53189db279c9637b7157d034de7ecd                     0.7s
 => => extracting sha256:91ff8760033c3b9bc9c3a3bcc6ff66c113ec9e13f94fa2f12ae4b221aae996b9                     0.1s
 => => extracting sha256:f3ba2250c52436e41e9f2f3830af92de466fcacc2d848264e1f9fdc76975d803                     0.4s
 => => extracting sha256:7ccd73948dde4b7f1623ac6bf9b58706fb83d0a363d765d7c799d46035ceb563                     0.0s
 => [internal] load build context                                                                             0.1s
 => => transferring context: 639B                                                                             0.0s
 => [2/4] WORKDIR /app                                                                                        0.2s
 => [3/4] RUN pip install --no-cache-dir matplotlib                                                          39.4s
 => [4/4] COPY . .                                                                                            0.1s
 => exporting to image                                                                                        5.6s
 => => exporting layers                                                                                       4.0s
 => => exporting manifest sha256:a973c2d753bd46af0a2296e5b20b70967dad8678934f116aa38346fb4a989a17              0.1s
 => => exporting config sha256:5901a916724141d1c86fbe77de3a511e69cf5e909ce23c95b1fde66eb74c0a82               0.1s
 => => exporting attestation manifest sha256:eacbae01df546d40c23706dc0854001bfb6ed76cdcc8c18070ff2f3a7987fdaf  0.1s
 => => exporting manifest list sha256:bb7863f122541255770706a89d9adb19e7a62609da25663ee1da84804bad1243         0.1s
 => => naming to docker.io/library/contenedor-python-grafico:latest                                           0.0s
 => => unpacking to docker.io/library/contenedor-python-grafico:latest                                        1.1s
nonbankstew@MSI:~/tarea-docker/contenedor-python-grafico$ docker run --rm -v "$PWD:/app" contenedor-python-grafico
Gráfico generado correctamente en output/grafico.png
nonbankstew@MSI:~/tarea-docker/contenedor-python-grafico$ ls output
grafico.png
nonbankstew@MSI:~/tarea-docker/contenedor-python-grafico$
nonbankstew@MSI:~/tarea-docker/contenedor-python-grafico$ explorer.exe output
nonbankstew@MSI:~/tarea-docker/contenedor-python-grafico$
```

Código en Python que utiliza la librería matplotlib para generar una gráfica a partir de datos simulados.

```
nonbankstew@MSI: ~/tarea-c
  GNU nano 7.2                    grafico.py
import matplotlib.pyplot as plt
import random

# Generar datos simulados
dias = ["Lun", "Mar", "Mié", "Jue", "Vie"]
valores = [random.randint(1, 10) for _ in dias]

plt.figure()
plt.plot(dias, valores, marker="o")
plt.title("Datos simulados de sensor (IoT)")
plt.xlabel("Días")
plt.ylabel("Lectura")
plt.grid(True)

plt.savefig("output/grafico.png")

print("Gráfico generado correctamente en output/grafico.png")


File to insert [from ./]:
^G Help          M-F New Buffer      ^X Execute Command
^C Cancel        M-N No Conversion   ^T Browse
```

Dockerfile del contenedor adicional, donde se define la imagen base, instalación de dependencias y ejecución del script en Python.

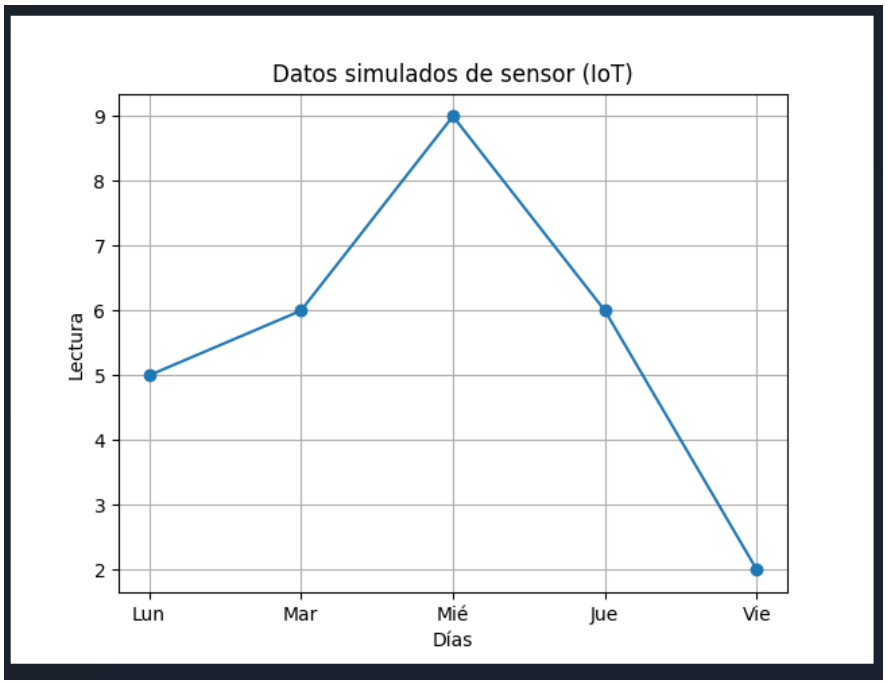
```
nonbankstew@MSI: ~/tarea-c
  GNU nano 7.2                    Dockerfile
FROM python:3.11-slim

WORKDIR /app

RUN pip install --no-cache-dir matplotlib

COPY . .

CMD ["python", "grafico.py"]

^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify
```

Gráfica generada por el contenedor en Python, representando datos simulados de un sensor IoT en formato visual.